IMPROVING THE GENERALIZATION ABILITY OF THE CHAOTIC TIME-SERIES CLASSIFICATION MODELS BY RESIDUAL COMPONENT EXTRACTION

Anonymous authors
Paper under double-blind review

ABSTRACT

This paper presents a new method in chaotic time-series classification that improves the generalization ability of the machine learning models. Such models are trained on a certain set of parameters of the nonlinear dynamical systems. Because of the sensitive dependence on initial conditions of such systems, when the parameters change the accuracy of machine learning models drops drastically. In our paper, we improve the generalization ability of the Multivariate LSTM-FCN model, by extracting and training the model on the residual component of the time-series. The steps applied are generic and can be reapplied to any other machine learning method. As an illustrative example, we provide a problem of the computation of the 2D bifurcation diagrams for the oscillatory time-series (periodic or chaotic) obtained as solutions for nonlinear systems of ordinary differential equations. The Multivariate LSTM-FCN network was trained on the time-series set obtained for the plane of parameters $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1$ and it achieved the accuracy of 96.96%. Once the parameters where changed to $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 2.8) \times 2$ the accuracy dropped to 34.38%. Whereas the accuracy of the model trained on residual components of the same dataset decreased from 96.50% to 65.52% for the new parameters time-series dataset. Furthermore, we show that by taking the mean and variance of the residual component we are able to extract many very interesting features of the 2D bifurcation diagrams using the KMeans, GMM, and OPTICS clustering algorithms.

1 INTRODUCTION

Time-series classification is a problem appearing widely in various scientific fields like climate prediction, biology, medical sciences, nuclear fusions, nanoscale engineering, embedded computation systems, secure communication, cryptography, and many others. A very important issue of the problem is the classification of the chaotic and periodic time-series solutions - the issue that emerged from the study of dynamical systems described by nonlinear ordinary differential equations (ODEs). The well-known dynamical systems are the Rössler, Lorenz, Hamiltonian and Chen systems, Chua circuit, and others.

Typical computational tools to analyze chaotic signals and systems, their sensitive dependence on initial conditions and continuous spectrum are, for example, the Lyapunov exponents, Fourier spectrum, return maps, bifurcation diagrams, statistical surrogate data methods in hypothesis testing, and others. It is important to apply those tools during the prototyping design phase in order to avoid surprising responses, for example, periodic ones when chaotic responses are needed in cryptography, classified communication, and coding of information. When parameters (i.e. electronic component values) change, then we often draw various bifurcation diagrams to determine how particular elements of the systems impact the overall dynamics. Two or more simultaneously changing parameters of the analyzed system present computational challenges in the process of creation and graphical visualization of the 2D (and nD) diagrams, as we need advanced hardware and software resources for that purpose Benettin et al. (1980); Gottwald & Melbourne (2004; 2016); Marszalek et al. (2019); Marszalek & Sadecki (2019a;b); Pentegov & Sidorets (1990).

The problem of generating 2D (and nD) bifurcation diagrams can be easily transformed into the time-series classification one whereby by applying the machine learning methods we can determine chaotic or periodic behavior for the desired set of parameters or initial values of the dynamical system. In recent years a number of results proposing how to solve such problems by using different types of machine learning algorithms have been published. Those algorithms are: the Recurrent Neural Networks (Cestnik & Abel (2019); Krishnaiah et al. (2006)), Extreme Learning Machines (Itoh & Adachi (2017); Itoh et al. (2020)), LSTM-FCN networks (Hassona et al. (2021)), Time Series Forests with Entropy (Deng et al. (2013)), LKCNN networks (Boullé et al. (2020)), and others. Although those models perform well on the training datasets, they very often tend to overfit tested on the new datasets as shown in Hassona et al. (2021); Boullé et al. (2020). These lead us to **the main objective of this paper**, which is improving the generalization abilities of the machine learning models in order to make them useable in real-life scenarios. The main problem of the classification of the chaotic time-series obtained from dynamical systems is the high dependence of these systems on the initial conditions. Changing even one of the parameters can lead to a completely new, unexpected response (time-series) of the system. This makes it extremely hard for machine learning methods to classify correctly new responses, that were learned on a limited amount of data for a certain set of parameters.

Boullé et al. (2020) attempted to improve the performance ability of the algorithms by advancing the architecture of the neural networks and applying a Large kernel convolutional neural network (LKCNN). The LKCNN model surpassed the ResNet, FCN MLP, and ShallowNet models trained on the dataset obtained from the logistic map and tested on a sine-circle map, dropping the accuracy of the LKCNN network only by 9%, while the other models' accuracy dropped even by 52%. However, in the present paper, we implement a different approach and show that applying the time-series decomposition and extracting the residual component from the time-series one can improve the generalization ability of the models as well. We also show that the residual component holds essential information that determines the chaotic behavior of the dynamical system. Also, we compare the performance of the MLSTM-FCN network (Karim et al. (2019)), giving as the inputs the original signals from the dynamical system versus using the residual components of the original signals as the inputs for different sets obtained by changing the parameters of the dynamical system. Next, by using only the mean and variance of the residual components of the time-series we show that the clustering algorithms can create very similar diagrams based on the partial information of the signal only. We also apply a simple algorithm to transform the clustering problem into a classification problem yielding very decent results, while having to use only a chunk of the label data, making it a lot more useful tool in the investigation of the dynamical systems than the conventional neural networks that need much more data to be able to classify the oscillatory time-series. This seems particularly useful and preferred in real-life applications over the above-mentioned algorithms.

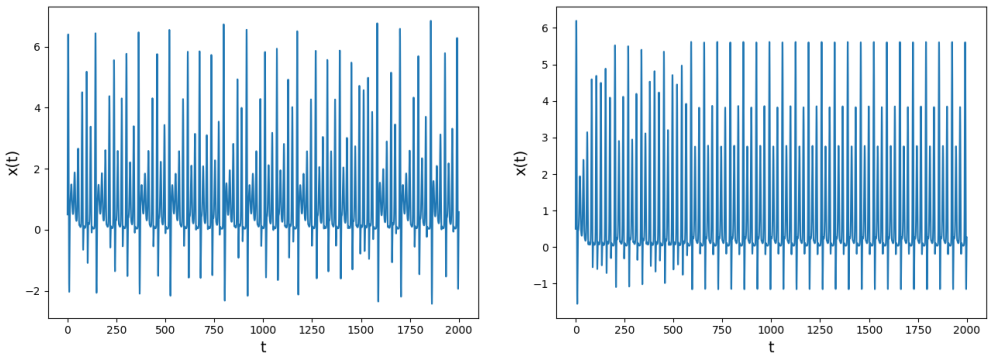
The paper is organized as follows. In Section 2 we describe our chaotic dynamical system (electric arc model) used for testing purposes and provide various methods of classifying time-series. The main results are presented in Section 3 with an extensive amount of illustrative material in the forms of 2D bifurcation diagrams and measures to evaluate the predictive performance of various ML algorithms. Comments on the obtained results and possible extensions of our research are included in Section 4.

2 Dynamical system, decomposition algorithm and machine learning methods used

2.1 Dynamical system and time-series data

For illustration purposes, we chose to study the oscillating electric arc RLC circuits (more in Appendix 1) described by a system of ordinary differential equations (Hassona et al. (2021))

$$\begin{aligned} \frac{dx}{dt} &= \frac{1}{L}(y - xz^{-1/3}) \\ \frac{dy}{dt} &= \frac{1}{RC}(R + 1 - y - Rx) \\ \frac{dz}{dt} &= x^2 - z \end{aligned}$$



(a) Chaotic time-series

(b) Periodic time-series for $t > 600$

Figure 1: Examples of (a) chaotic time-series for $(C, R) = (3.279800, 15.484140)$, and (b) periodic time-series for $(C, R) = (3.312855, 6.570952)$.

The above ODE and the method used to obtain the original time-series responses (described below) are taken from Hassona et al. (2021). For further analysis we consider 2D parameter space of $(C, R) \in [2.8, 3.4] \times [6.0, 15.5]$ with $L = 1$. To solve the above system we used the 4th-order explicit Runge–Kutta (RK-4) method with the time interval $0 \leq t \leq 2000$, integration step $dt = 0.01$ and initial conditions $[x(0), y(0), z(0)] = [0.5, 4.0, 1.0]$. After solving the above ODEs for the pair (C, R) , we identify of local maximum points of the obtained oscillatory solution to classify each time-series as chaotic or periodic. Identification of the local maximum points for the obtained time-series was done in a time interval $[t_{min}, t_{max}]$, using a simple algorithm based on the three consecutive discrete values, say $x(t_{k-1})$, $x(t_k)$ and $x(t_{k+1})$, see [8]. The identification was done backwards from $t_{max} = 2000$ to $t_{min} = 1000$. The step size dt in the RK-4 solver was selected such that in the case of periodic solutions $dt << T$, where T was the minimum period of oscillations in the assumed rectangular area (C, R). The value of $dt = 0.01$ was proved to be small enough for the intervals of parameters R and C given above. Once the maximum points are identified for $t_{min} \leq t \leq t_{max}$, then the type of periodic response is established as period-n oscillations, $n = 1, 2, \ldots, 22$. In our examples in this paper, all responses with $n \geq 23$ are collected together as one class of responses and, making a rough assumption, given the status of the chaotic responses class. There is nothing special about the number 23, as one can easily consider a bigger value, say a few hundred or even a thousand. Examples of chaotic and steady-state periodic time-series are shown in Fig.2. More on the method can be found in [8]. One run of the numerical solver in the above method results in $600,003$ data points (discrete solution values) for the three variables in the system. Data labels (with a color attached to each pair of points (C_i, R_j)) were generated, as shown in Fig.2(a).

2.2 Training data

The method mentioned in Section 2.1 (Marszalek et al. (2019); Marszalek & Sadecki (2019a)) results, for the given parameters, in $600,003$ data points and labels for each of the three variables in dynamical system. This method uses the specific properties of trajectories of the dynamical systems. The trajectories of the system evolve around the stationary point $(x, y, z) = (1, 1, 1)$. The method considers the projection of those trajectories to the XY plane, and counts the number of crossings of a line from the projected stationary point $(x, y) = (1, 1)$, by the projected trajectories, see Fig. 3.

We used all three variables for our calculations (for each pair of $180,000$ points (C_i, R_j) in the C and R parameter space), although to determine the chaotic behavior, it is enough to study only one variable. We used a rather rough assumption that the identification process of periodic responses was done up to the number of local maximum points equal to 22. If the number of maximum values happened to be greater or equal to 23 in one period, then all such responses were labeled with $n = 23$. Periodic responses with the number of local maximum points n from 1 to 22 were classified

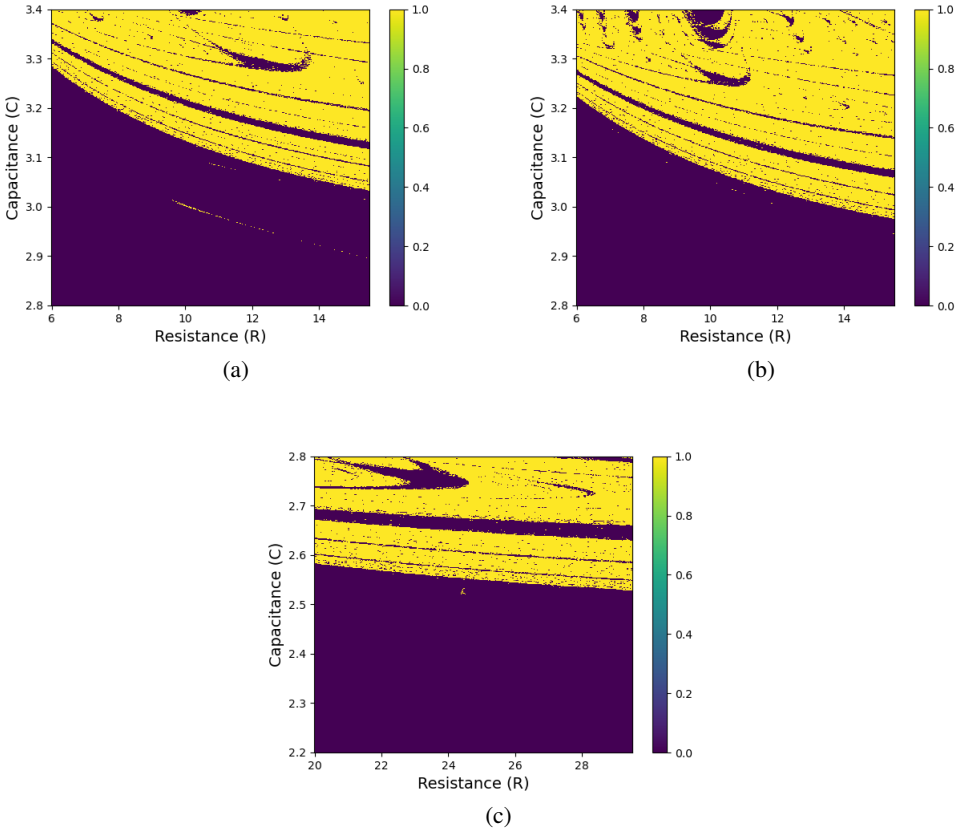


Figure 2: Bifurcation diagram generated using the method described in Marszalek & Sadecki (2019a) for parameters (a) $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1$, (b) $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1.1$, and (c) $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 2.8) \times 2$. Dark violet regions indicate periodic dynamics, whereas the yellow color corresponds to chaotic ones.

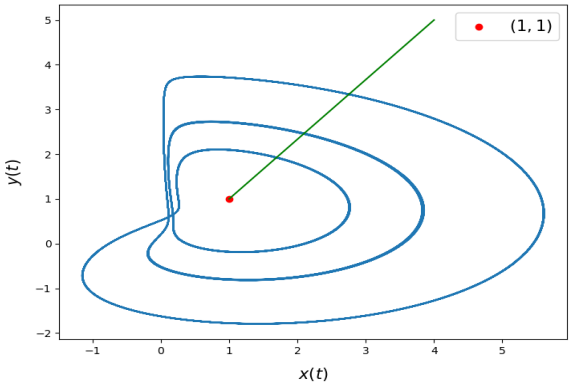


Figure 3: The method used to calculate the labels for time-series. The blue curves denote the trajectories of the system and the red point is the projected stationary point. The number of crossings of the green line by the blue trajectories gives the number of local maximum of the examined time-series.

as periodic ones (named period-n oscillations). If a response was identified as not being periodic in the interval $0 \leq t \leq 2000$, then such a response was classified as a chaotic one and labeled as being $n = 23$. The number of local maximum points heavily depends on the integration step dt used to compute solutions of the system. If it is too small, the trajectories start to overlap each other, so it is impossible to correctly count the number of crossings. On the other hand, if it is too big, then the time of calculation increases. In the assumed parameter space for C and R there were neither unstable nor constant steady-state responses identified.

To train the MLSTM-FCN network we used time-series data in the interval of $1800 \leq t \leq 2000$, which results in $20,001$ data points. We took only every 8 point of the series, so the obtained time-series had 2501 points. It was convenient to take 23 of local maximum points as a threshold between the periodic and chaotic classes. The data generation, as well as the model training and classification, was performed on Tesla K80 GPU using Python framework PyTorch. As for time-series decomposition, we used library $seasonal$ and performed calculations on the Intel(R) Xeon(R) CPU ES-2640 v2 processor at 2.00 GHz. The same setup was used for running the clustering algorithms (KMeans Jirong et al. (2009), GMM Reynolds & Rose (1995), and OPTICS Ankerst et al. (1999)), although we used standard Scikit-Learn. For the purpose of training the MLSTM-FCN network, we divided the data set into the training, validation, and test sets — respectively at 70%, 15%, and 15% of the whole data set, which resulted in 100836 time-series samples for training set, and 21608 for both validation and test sets. As for the clustering algorithms, after calculating the mean and variance of the residual components, we used the whole data set to cluster it. The code needed to reproduce our experiments and data is provided with the supplementary material.

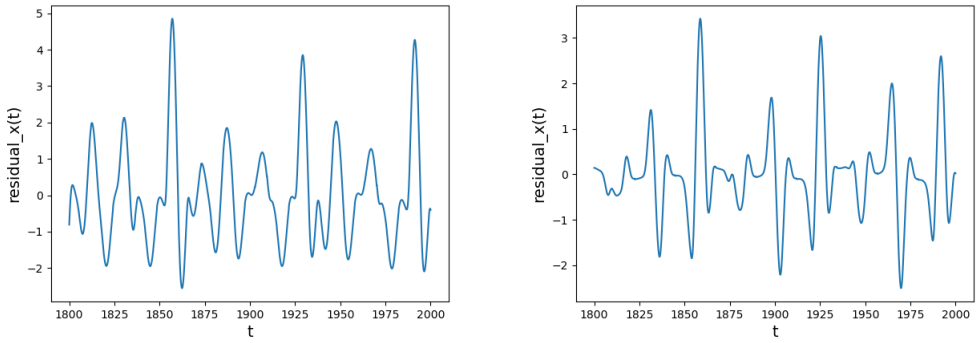
2.3 Decomposition algorithm

There are many decomposition algorithms but most of them require prior knowledge of the periodicity of the time-series, which in the case of chaotic dynamical systems is almost impossible to anticipate. Therefore, one can use either the decomposition approach (i.e. the empirical mode decomposition Shi et al. (2017)) which doesn't require prior knowledge of the periodicity of the time-series. In this paper, to estimate the period, we used the periodogram averaging in our calculations. Firstly, one should detrend the data. We used the coarse-fitted cubic spline to do that. However, one could also use the mean or median filters or any other way to obtain the trend. In the next step, we estimate the range of likely periods for our time-series data by using the fast and robust periodogram averaging technique, developed by (Welch (1967)). Then we tested the obtained periodicity values for best fit to the detrended data using the cross-validated residual errors and R^2 of the leave-one-out cross-validation. Finally, we removed the seasonal variation of the dominant frequency using the estimated period. More about Welch's method can be found in Appendix 2. Once, we remove the trend and seasonal components we obtain the residual component, in Fig. 4 we show the residual components of the time series presented in Fig. 2.

2.4 Machine learning methods applied

We used the state-of-the-art Multivariate LSTM-FCN network, although one could also use its univariate LSTM-FCN version. This way we supply as much information about the dynamical system as possible, but it is well-known that each of the signals behaves the same way, meaning that if one of them is a chaotic time-series, the other ones will also behave in a chaotic way. The same applies for the periodic cases of the time-series. In the second part of our experiments, we applied clustering methods, namely the KMeans algorithm, Gaussian Mixture Models, and OPTICS algorithm - each of them representing different types of clustering algorithms, that is, respectively, the centroid-based clustering, distribution-based clustering, and density-based clustering. We also turned the clustering problem into a classification problem, applying a very simple algorithm to estimate the accuracy of the clustering algorithms:

1. For each cluster, we took 5 sample time-series data, and we labeled them accordingly: chaotic time-series were denoted as 1, whereas periodic as 0.
2. We took a mean of the 5 labeled data obtained from 1., if the mean > 2 we classified all time-series in a cluster as a chaotic time-series, otherwise we classified them as periodic responses.



(a) Residual component of chaotic time-series (b) Residual component of periodic time-series

Figure 4: Examples of residual components of (a) chaotic time-series for $(C, R) = (3.279800, 15.484140)$, and (b) periodic time-series for $(C, R) = (3.312855, 6.570952)$ after pre-processing.

3. Finally we obtained sets of labeled data for each cluster, which allowed us to calculate the overall accuracy of labeled time-series data.

More about the applied methods can be found in Appendix 3.

2.5 MLSTM-FCN NEURAL NETWORKS TRAINING AND CLUSTERING PARAMETERS

Training of each neural network (MLSTM-FCN and MLSTM-FCN Residual - network trained on residual components, **please do not confuse it with the residual networks known as ResNets**) was performed for 200 epochs until the validation accuracy saturated around 96%. Throughout the training process, we used Adam optimizer (Kingma & Ba (2015)) with a learning rate set to 0.0007. As for the KMeans algorithm, we used 23 clusters with a random state equal 0 and a maximum of iterations equal to 1000. As for the parameters of Gaussian Mixture Model, we used 23 components with covariance type $full$, and the OPTICS algorithm was left to default values provided by the Scikit-Learn library.

3 THE RESULTS

The results of the MLSTM-FCN and MLSTM-FCN Residual networks are presented in the Tables 1, 2, and 3, and in Fig. 5. As expected we obtained very high-performance metrics for both MLSTM-FCN neural networks, with slightly better accuracy of MLSTM-FCN over MLSTM-FCN Residual. However, if we change the parameter L of the dynamical system to $L = 1.1$, as expected, the performance of both networks declined, but MLSTM-FCN Residual started performing better than MLSTM-FCN. When we changed the parameters R, C and L to $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 2.8) \times 2$, the accuracy of MLSTM-FCN network drastically dropped from 96.96% to 34.38%, whereas accuracy of MLTM-FCN Residual decreased from 96.50% to 65.52%. It's worth noting that if we compare both bifurcation diagrams from Fig. 2(b), and Fig. 2(c) with that of Fig. 5 (c, d, e and f), the MLSTM-FCN Residual model is apparently better.

As for the clustering algorithms, the results are very interesting. The results are present in Table 4 and in Fig. 6. It was sufficient to take only the variance and mean of the residual components to obtain clusters with very similar features that are present in the diagram in Fig. 2(a). Furthermore, the simple algorithm that was transforming clusters into labels (based on 5 labels for each cluster) performed very well. In fact, one should compare the metrics of the MLSTM-FCN networks for

$L = 1.1$, with the KMeans and GMM algorithms as both give very little insight into the true labels of the time-series data. The KMeans and GMM performed almost as well as neural networks, both having over 90% accuracy, whereas neural networks gave an accuracy of about $92 - 93\%$. However, if we compare them to the metrics of the MLSTM-FCN networks for $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 2.8) \times 2$, then all three algorithms outperformed both neural networks.

4 CONCLUSIONS

In this paper, we showed that the time-series decomposition helps in the improvement of the classification abilities of the MLTSM-FCN neural networks. Moreover, we showed that the residual components transfer most of the information about the chaotic behavior of the dynamical system, as the clustering algorithms were able to extrapolate a lot of features of the bifurcation diagram without prior knowledge of either of the dynamical system itself or labels.

Using the residual component has a huge advantage over the original signal, it does not contain information about the trend and seasonal component of the time-series. For humans, it is intuitive that in order to differentiate chaotic time-series from the periodic one we search for the "noise" that such chaotic signals contain. In the same way, residual components, in theory, should contain only the "noise" of the signal, so if the "noise" is high - the residual component has high variance and mean - one could classify it as chaotic, and vice-versa if there is a little of "noise" - the residual component has small variance and mean - one could classify it as a periodic signal.

We currently study the abilities of the clustering algorithms to help with the time-series classification and feature extraction of the chaotic time- series. The presented results pertaining to the clustering algorithms are our preliminary studies of them in the context of time-series classification, yet our preliminary results are very promising, and we hope for better and more advanced results in the near future.

Table 1: Performance metrics for $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1$

	Acc.	**Prec.**	**Recall**	**F1 Score**	**ROC AUC**	**PR AUC**
MLSTM-FCN	96.96	94.42	98.21	96.28	94.02	89.01
MLSTM-FCN Residual	96.50	95.46	95.82	95.64	94.02	89.01

Table 2: Performance metrics for $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1.1$

	Acc.	**Prec.**	**Recall**	**F1 Score**	**ROC AUC**	**PR AUC**
MLSTM-FCN	92.43	86.70	99.56	92.69	91.51	85.18
MLSTM-FCN Residual	93.28	89.61	97.35	93.32	91.51	85.18

Table 3: Performance metrics for $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 3.8) \times 2$

	Acc.	**Prec.**	**Recall**	**F1 Score**	**ROC AUC**	**PR AUC**
MLSTM-FCN	34.38	33.16	100.00	49.80	93.11	78.95
MLSTM-FCN Residual	65.52	48.55	99.32	65.22	93.11	78.95

Table 4: Performance metrics for clustering methods with applied algorithm from section 2.4 where $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1$

	Acc.	**Prec.**	**Recall**	**F1 Score**
KMeans	91.41	82.83	99.07	90.22
GMM	90.08	80.78	98.69	88.84
OPTICS	67.09	82.42	22.57	35.43

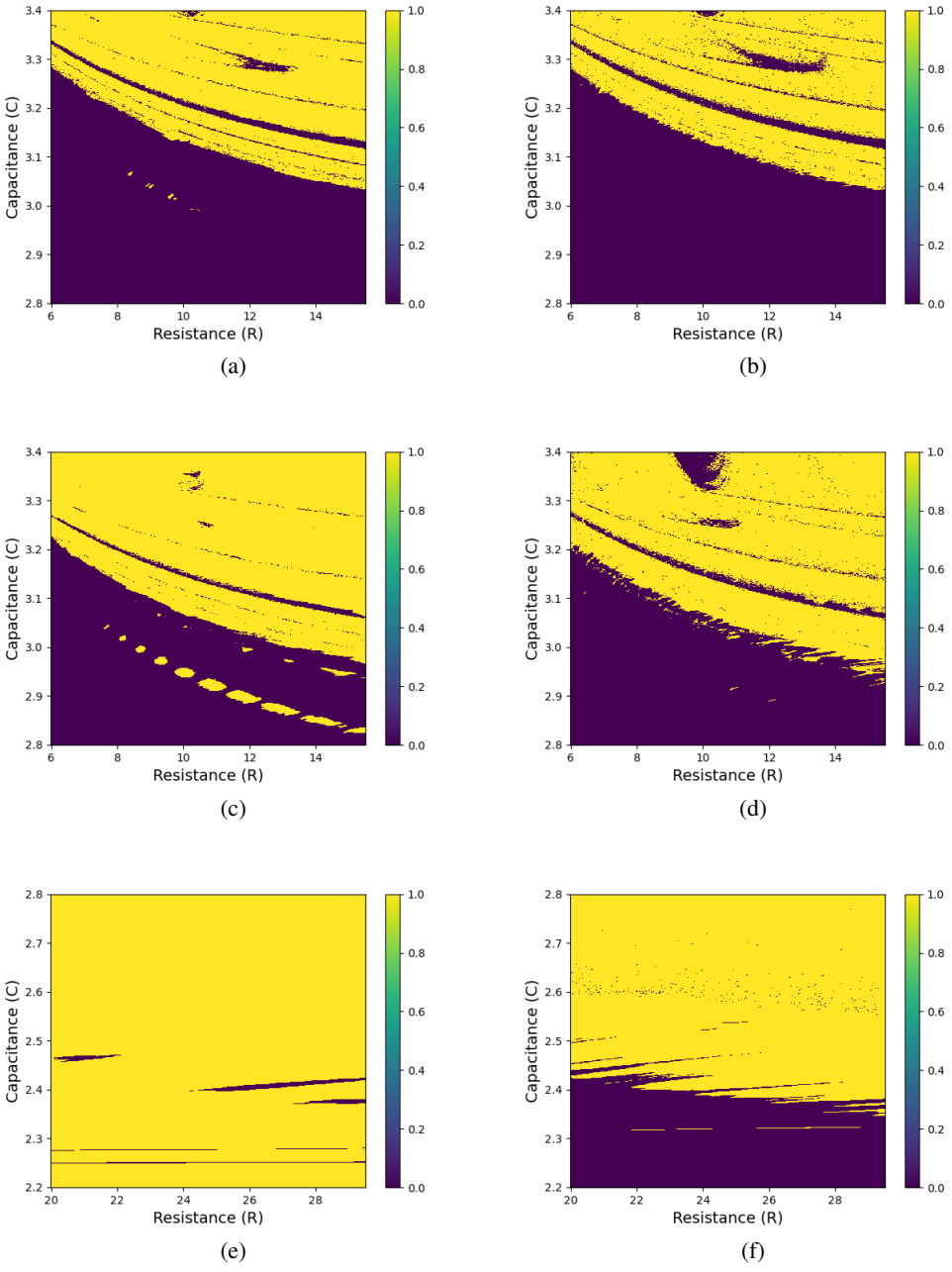
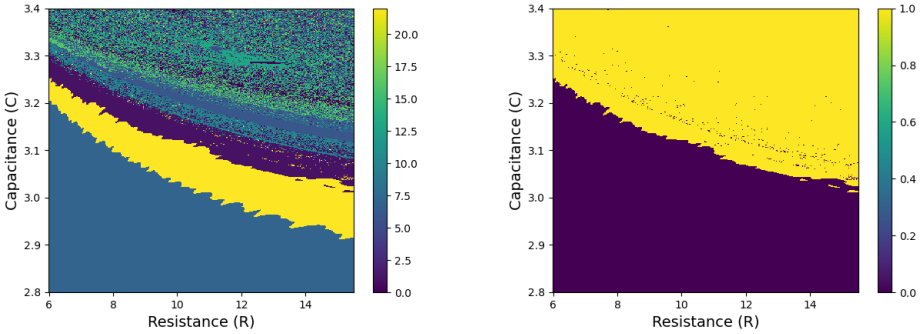
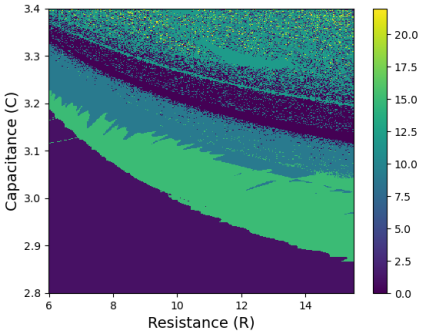


Figure 5: Reconstructed bifurcation using MLSTM-FCN (left side) and MLSTM-FCN Residual (right side) diagrams for (a and b) $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1$, (c and d) $(R_1, R_2) \times (C_1, C_2) \times L = (6, 15.5) \times (2.8, 3.4) \times 1.1$, and (c and d) $(R_1, R_2) \times (C_1, C_2) \times L = (20, 29.5) \times (2.2, 2.8) \times 2$. Regions in the reconstructed bifurcation diagrams with dark violet color indicate periodic dynamics, whereas the yellow color corresponds to chaotic ones.

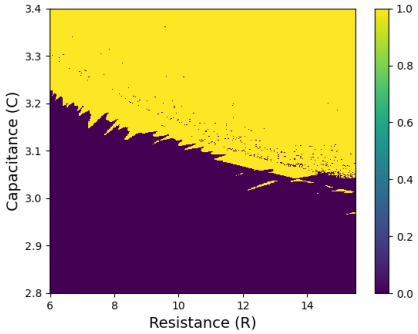


(a)

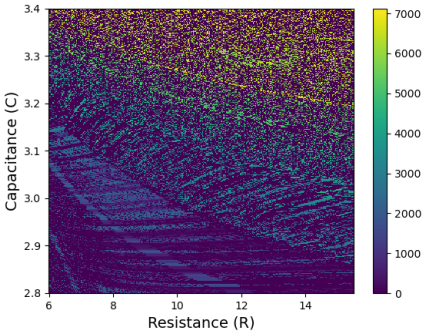
(b)



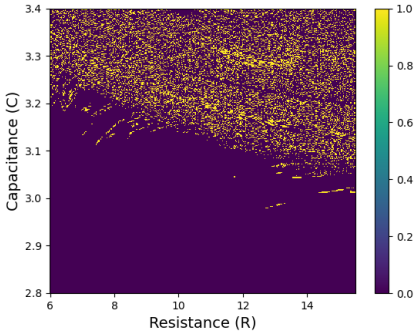
(c)



(d)



(e)



(f)

Figure 6: Diagrams obtained by clustering the mean and variance of residual components for $L = 1$ (left side) and reconstructed bifurcation diagrams, obtained by applying algorithm described in section 2.4 (right side) using (a and b) KMeans, (c and d) Gaussian Mixture Models, and (e and f) OPTICS algorithm. Regions with same colors in diagrams obtained by clustering algorithms represents particular cluster, components, density respectively for each algorithm. Regions in the reconstructed bifurcation diagrams with dark violet color indicate periodic dynamics, whereas the yellow color corresponds to chaotic ones.

REFERENCES

M. Ankerst, M.M. Breunig, H.-P. Kriegel, and J. Sander. OPTICS: Ordering Points to Identify the Clustering Structure. *SIGMOD Record (ACM Special Interest Group on Management of Data)*, 28(2):49–60, 1999. doi: 10.1145/304181.304187.

G. Benettin, L. Galgani, A. Giorgilli, and J.-M. Strelcyn. Lyapunov Characteristic Exponents for smooth dynamical systems and for hamiltonian systems; A method for computing all of them. Part 2: Numerical application. *Meccanica*, 15(1):21–30, 1980. doi: 10.1007/BF02128237.

N. Boullé, V. Dallas, Y. Nakatsukasa, and D. Samaddar. Classification of chaotic time series with deep learning. *Physica D: Nonlinear Phenomena*, 403:132261, 2 2020. ISSN 01672789. doi: 10.1016/j.physd.2019.132261. URL https://linkinghub.elsevier.com/retrieve/pii/S0167278919304737.

R. Cestnik and M. Abel. Inferring the dynamics of oscillatory systems using recurrent neural networks. *Chaos*, 29(6), 2019. doi: 10.1063/1.5096918.

H. Deng, G. Runger, E. Tuv, and M. Vladimir. A time series forest for classification and feature extraction. *Information Sciences*, 239:142–153, 2013. doi: 10.1016/j.ins.2013.02.030.

G.A. Gottwald and I. Melbourne. A new test for chaos in deterministic systems. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 460(2042):603–611, 2004. doi: 10.1098/rspa.2003.1183.

G.A. Gottwald and I. Melbourne. *The 0-1 test for chaos: A review*, volume 915. 2016. doi: 10.1007/978-3-662-48410-4{_}7.

S. Hassona, W. Marszalek, and J. Sadecki. Time series classification and creation of 2D bifurcation diagrams in nonlinear dynamical systems using supervised machine learning methods. *Applied Soft Computing*, 113, 2021. doi: 10.1016/j.asoc.2021.107874.

Y. Itoh and M. Adachi. Reconstruction of bifurcation diagrams using an extreme learning machine with a pruning algorithm. In *Proceedings of the International Joint Conference on Neural Networks*, volume 2017-May, pp. 1809–1816, 2017. ISBN 9781509061815. doi: 10.1109/IJCNN.2017.7966070.

Y. Itoh, S. Uenohara, M. Adachi, T. Morie, and K. Aihara. Reconstructing bifurcation diagrams only from time-series data generated by electronic circuits in discrete-time dynamical systems. *Chaos*, 30(1), 2020. doi: 10.1063/1.5119187.

G. Jirong, Z. Jieming, and C. Xianwei. An enhancement of K-means clustering algorithm. In *2009 International Conference on Business Intelligence and Financial Engineering, BIFE 2009*, pp. 237–240, 2009. ISBN 9780769537054. doi: 10.1109/BIFE.2009.204.

F. Karim, S. Majumdar, H. Darabi, and S. Harford. Multivariate LSTM-FCNs for time series classification. *Neural Networks*, 116:237–245, 2019. doi: 10.1016/j.neunet.2019.04.014.

D.P. Kingma and J.L. Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, 2015.

J. Krishnaiah, C.S. Kumar, and M.A. Faruqi. Modelling and control of chaotic processes through their Bifurcation Diagrams generated with the help of Recurrent Neural Network models: Part 1-simulation studies. *Journal of Process Control*, 16(1):53–66, 2006. doi: 10.1016/j.jprocont.2005.04.002.

W. Marszalek and J. Sadecki. Complex Two-Parameter Bifurcation Diagrams of a Simple Oscillating Circuit. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 66(4):687–691, 2019a. doi: 10.1109/TCSII.2018.2871063.

W. Marszalek and J. Sadecki. Parallel computing of 2-D bifurcation diagrams in circuits with electric arcs. *IEEE Transactions on Plasma Science*, 47(1):706–713, 2019b. doi: 10.1109/TPS.2018.2871576.

W. Marszalek, H. Podhaisky, and J. Sadecki. Computing two-parameter bifurcation diagrams for oscillating circuits and systems. *IEEE Access*, 7:115829–115835, 2019. doi: 10.1109/ACCESS.2019.2936175.

I.V. Pentegov and V.N. Sidorets. Energy parameters in a mathematical model of a dynamic welding arc. *Welding International*, 4(4):272–275, 1990. doi: 10.1080/09507119009447722.

D.A. Reynolds and R.C. Rose. Robust Text-Independent Speaker Identification Using Gaussian Mixture Speaker Models. *IEEE Transactions on Speech and Audio Processing*, 3(1):72–83, 1995. doi: 10.1109/89.365379.

Y.-H. Shi, Y. Leng, Y.-K. Yang, H.-X. Wang, and S. Ge. A sinusoidal-signal-assisted method of improving multivariate empirical mode decomposition. In *2016 13th International Computer Conference on Wavelet Active Media Technology and Information Processing, ICCWAMTIP 2017*, pp. 350–353, 2017. ISBN 9781509061259. doi: 10.1109/ICCWAMTIP.2016.8079871.

P.D. Welch. The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967. doi: 10.1109/TAU.1967.1161901.

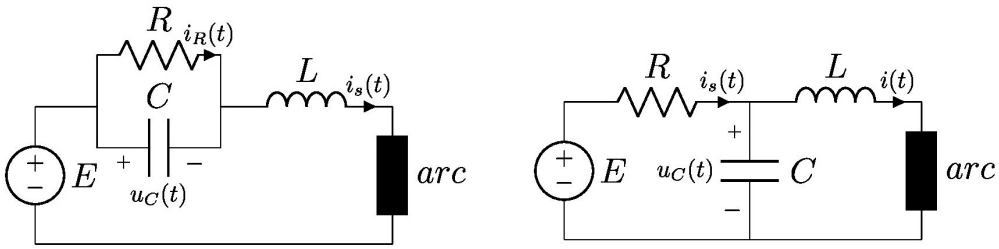
A APPENDIX 1

The following ODE model is used in all our computations

$$\begin{array}{rcl} \frac{di}{d\tau} & = & \frac{1}{L}(u_C - \frac{U(i_\theta)}{i_\theta} i) \\ \frac{du_C}{d\tau} & = & \frac{1}{RC}(E - u_C - Ri) \\ \frac{di_\theta^2}{d\tau} & = & \frac{1}{\theta}(i^2 - i_\theta^2) \end{array} \quad \rightarrow \quad \begin{array}{rcl} \frac{dx}{dt} & = & \frac{1}{L}(y - xz^m) \\ \frac{dy}{dt} & = & \frac{1}{RC}(R + 1 - y - Rx) \\ \frac{dz}{dt} & = & x^2 - z \end{array} \tag{1}$$

The i and u_C are the current through L and voltage across C, respectively. The i_s and i_θ stand for the *source* and arc current.

The dimensionless ODE model in section 2.1 (see above) describes each of the two circuits shown below in Fig.7. The nonlinear arc element is described by the *static* voltage-current characteristic $U(i_\theta) = U_0(i_\theta/I_0)^m$, where U_0 and I_0 are two positive constants. The $U(i_\theta)$ and m are the arc voltage and negative constant (it depends on the type of electric arc), respectively. We used $m = -1/3$ in all our calculations. Also, the time dependent variables are $x = i/I_0$, $y = u_C/U_0$ and $z = i_\theta^2/I_0^2$, while the R, L and C are the resistance, inductance, and capacitance, respectively. We use various pairs of the triple (R, L, C) in our two-parameter bifurcation diagrams in this paper.



(a) Circuit A

(b) Circuit B

Figure 7: Electric arc circuits. Circuit B is also described by dynamical system (1) with a suitable change of variables.

B APPENDIX 2

B.1 WELCH METHOD AND PERIODOGRAMS

The periodogram concept is based on the definition of the power spectral density (PSD). Let $x_w(n) = w(n)x(n)$ indicate a windowed segment of samples from a random process $x(n)$, where the window function w (classically the rectangular window) contains M nonzero samples. Then the periodogram is defined as the squared magnitude of the Discrete Time Fourier Transform (DTFT) of x_w, divided by M.

$$P_{x,M}(\omega) = \frac{1}{M}|DTFT(x_w)|^2 = \frac{1}{M}\left|\sum_{n=0}^{M-1} x_w(n)e^{-j\omega n}\right|^2$$

In another words, the periodogram is the Fourier transform of the auto-correlation function.

The Welch method for estimating power spectra is performed by dividing the time signal into successive blocks, forming the periodogram for each block, and averaging.

Denote the m-th windowed, zero-padded frame from the signal x by

$$x_m(n) = w(n)x(n + mR), \quad n = 0, 1, \ldots, M-1, \ m = 0, 1, \ldots, K-1,$$

where R is defined as the window hop size, and let K denote the number of available frames. Then the periodogram of the m-th block is given by

$$P_{x_m,M}(\omega_k) = \frac{1}{M} \left|\text{FFT}_{N,k}(x_m)\right|^2 = \frac{1}{M} \left| \sum_{n=0}^{N-1} x_m(n) e^{-j2\pi nk/N} \right|^2$$

as before, and the Welch estimate of the power spectral density is given by

$$\hat{S}_x^W(\omega_k) \frac{1}{K} \sum_{m=0}^{K-1} P_{x_m,M}(\omega_k).$$

In other words, it's simply the average of periodograms across time.

B.2 Applying the Welch method to decompose time-series data

We used the same representation as in the Holt-Winters additive method, where the seasonality is represented as an offset for each seasonal interval. Periodicity of the time-series we applied the Welch method to estimate a range of likely periods. Then we tested the obtained periodicity values for best fit to the detrended data using the cross-validated residual errors and R^2 of the leave-one-out cross-validation. Once we estimated the period, we could decompose the time-series by removing the seasonal variation of the dominant frequency. As mentioned in section 2.4 to detrend the time-series data we used the coarse fitted cubic splines. After removing the trend and seasonality, we obtained the residual components, which were used in further calculations.

C Appendix 3

C.1 MLSTM-FCN Networks

For numerous time-series datasets benchmarks, the Multivariate LSTM Fully Convolution Network is one of the state-of-the-art models [13]. The network is composed of two blocks, Fully Convolutional Network and LSTM network.

The Fully Convolutional Network is comprised of the Temporal Convolutions used as a feature extractors and the Global Average Pooling is used to reduce the number of parameters in the model prior to classification. A time-series signal is typically used as the input to the Temporal Convolutional Network.

The three stacked temporal convolutional blocks with filter sizes of 128, 256 and 128 make up the fully convolutional block. Each convolutional block is identical to the CNN architecture's convolution block. Each block begins with a temporal convolutional layer, followed by batch normalization and ReLU activation function. The Squeeze and Excite blocks come behind the first two blocks, and they improve the quality of representations produced by a network by explicitly modeling the interdependencies between the channels of convolutional features. The existence of Squeeze and Excite blocks is the main difference between LSTM-FCN and MLSTM-FCN networks. Finally, a global average pooling is applied following the final convolution block.

The time-series input is also processed through a dimension shuffle layer before being sent to the LSTM block, which is followed by a dropout. The long short-term memory recurrent neural networks (LSTM RNNs) are a step forward from general recurrent neural networks, which suffer from the vanishing gradient problem. By adding gating functions into their state dynamics, the LSTM RNNs overcome the vanishing gradient problem that affects traditional recurrent neural networks. The LSTM keeps track of state changes and outputs with a hidden vector and a memory vector at each time step.

Finally, the outputs of the global pooling layer and the LSTM block are combined and sent to a softmax classification layer, as shown in Fig.8.

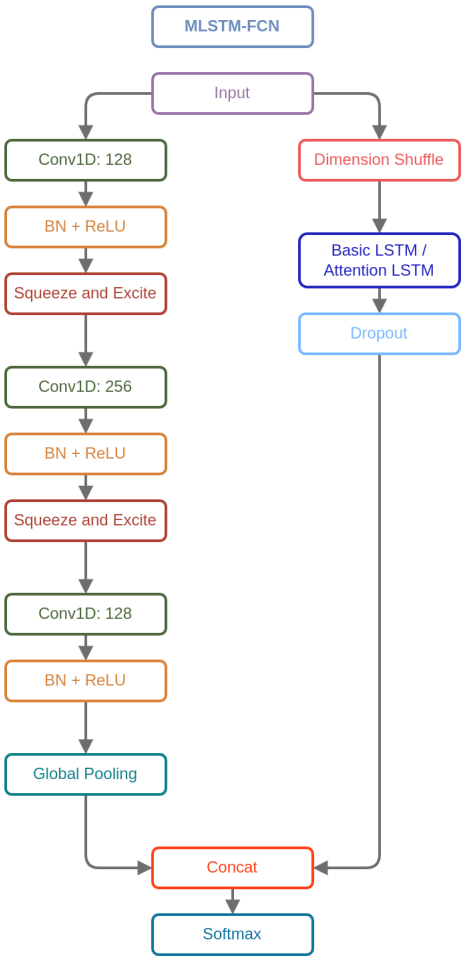


Figure 8: The MLSTM-FCN architecture. LSTM cells can be replaced by the Attention LSTM cells to construct the MALSTM-FCN architecture.

C.2 CLUSTERING ALGORITHMS

C.2.1 CENTROID-BASED CLUSTERING. THE KMEANS ALGORITHM.

In centroid-based clustering, each cluster is represented by a central vector, which is not necessarily a member of the data set. The most extensively used centroid-based clustering algorithm is the KMeans algorithm. The n data objects are separated into k partitions ($k << n$) using this polythetic hard clustering technique, with each partition representing a cluster. At least one data point must be present in each cluster. Additionally, each data object must belong to one group only. Furthermore, observations of the same cluster should be similar to each other. Objects from distinct groups, on the other hand, must be dissimilar. The KMeans algorithm's purpose is to maximize the distance between the centers of each pair of clusters while minimizing the distance between observations within each cluster. (e.g., minimize the sum of squared error, SSE, within a cluster.).

C.2.2 DISTRIBUTION-BASED CLUSTERING. THE GAUSSIAN MIXTURE MODEL.

Distribution models are the most directly linked clustering model to statistics. Objects most likely belonging to the same distribution can subsequently be simply identified as clusters. This method has the advantage of closely resembling how artificial data sets are created: by sampling random objects from a distribution.

The Gaussian Mixture Model is a semi-parametric soft clustering approach in which each cluster corresponds to a generative model aimed at discovering the parameters of a probability distribution (e.g., mean, covariance, density function) for that cluster. The process of learning is fitting a gaussian model to the data points. The Gaussian Mixture model assumes that clusters in n-dimensional space are distributed in a normal distribution.

C.2.3 DENSITY-BASED CLUSTERING. THE OPTICS ALGORITHM.

In the density-based clustering, the dense regions in the data space are separated from regions with lower density. If a cluster's density in a certain location exceeds a predefined threshold, observations are assigned to it.

The local density around a given observation in one cluster must reach a certain threshold. The radius ϵ of the circle that contains a specified number of neighbors around a given point and the minimum number of points $minPts$ around that radius define the local density.

The OPTICS algorithm extends DBSCAN, makes it less sensitive to parameter setting and finding structures among clusters. The intuition is that, based on two parameters, the higher density regions will be processed first before the lower ones, as follows:

0. Core distance: the smallest radius ϵ that contains at least $minPts$ observations.

1. Reachability distance: the minimum distance between two observations that makes them density-reachable.

With that, the OPTICS arranges observations into ordered clusters based on their density structure. Furthermore, it uses the computed reachability distance values for all points as a threshold to distinguish data from outliers.